

Aggregation is a “has a” or “part-whole” relationship: one class contains objects of another class. Aggregation is represented in UML class diagrams by an open diamond pointing to the “whole” part of the part-whole pair. Composition is a strong form of aggregation. Its arrowhead is solid rather than open.

Inheritance permits the reusability of software: Derived classes can extend the capabilities of base classes with no need to modify—or even access the source code of—the base class. This leads to new flexibility in the software development process, and to a wider range of roles for software developers.

Questions

Answers to these questions can be found in Appendix G.

1. Inheritance is a way to
 - a. make general classes into more specific classes.
 - b. pass arguments to objects of classes.
 - c. add features to existing classes without rewriting them.
 - d. improve data hiding and encapsulation.
2. A “child” class is said to be _____ from a base class.
3. Advantages of inheritance include
 - a. providing class growth through natural selection.
 - b. facilitating class libraries.
 - c. avoiding the rewriting of code.
 - d. providing a useful conceptual framework.
4. Write the first line of the specifier for a class `Bosworth` that is publicly derived from a class `Alphonso`.
5. True or false: Adding a derived class to a base class requires fundamental changes to the base class.
6. To be accessed from a member function of the derived class, data or functions in the base class must be `public` or _____.
7. If a base class contains a member function `basefunc()`, and a derived class does not contain a function with this name, can an object of the derived class access `basefunc()`?
8. Assume that the classes mentioned in Question 4 and the class `Alphonso` contain a member function called `alfunc()`. Write a statement that allows object `BosworthObj` of class `Bosworth` to access `alfunc()`.

9. True or false: If no constructors are specified for a derived class, objects of the derived class will use the constructors in the base class.
10. If a base class and a derived class each include a member function with the same name, which member function will be called by an object of the derived class, assuming the scope-resolution operator is not used?
11. Write a declarator for a no-argument constructor of the derived class `Bosworth` of Question 4 that calls a no-argument constructor in the base class `Alphonso`.
12. The scope-resolution operator usually
 - a. limits the visibility of variables to a certain function.
 - b. tells what base class a class is derived from.
 - c. specifies a particular class.
 - d. resolves ambiguities.
13. True or false: It is sometimes useful to specify a class from which no objects will ever be created.
14. Assume that there is a class `Derv` that is derived from a base class `Base`. Write the declarator for a derived-class constructor that takes one argument and passes this argument along to the constructor in the base class.
15. Assume a class `Derv` that is privately derived from class `Base`. An object of class `Derv` located in `main()` can access
 - a. public members of `Derv`.
 - b. protected members of `Derv`.
 - c. private members of `Derv`.
 - d. public members of `Base`.
 - e. protected members of `Base`.
 - f. private members of `Base`.
16. True or false: A class `D` can be derived from a class `C`, which is derived from a class `B`, which is derived from a class `A`.
17. A class hierarchy
 - a. shows the same relationships as an organization chart.
 - b. describes “has a” relationships.
 - c. describes “is a kind of” relationships.
 - d. shows the same relationships as a family tree.
18. Write the first line of a specifier for a class `Tire` that is derived from class `Wheel` and from class `Rubber`.

19. Assume a class `Derv` derived from a base class `Base`. Both classes contain a member function `func()` that takes no arguments. Write a statement to go in a member function of `Derv` that calls `func()` in the base class.
20. True or false: It is illegal to make objects of one class members of another class.
21. In the UML, inheritance is called _____.
22. Aggregation is
 - a. a stronger form of instantiation.
 - b. a stronger form of generalization.
 - c. a stronger form of composition.
 - d. a “has a” relationship.
23. True or false: the arrow representing generalization points to the more specific class.
24. Composition is a _____ form of _____.

Exercises

Answers to starred exercises can be found in Appendix G.

- *1. Imagine a publishing company that markets both book and audiocassette versions of its works. Create a class `publication` that stores the title (a string) and price (type `float`) of a publication. From this class derive two classes: `book`, which adds a page count (type `int`), and `tape`, which adds a playing time in minutes (type `float`). Each of these three classes should have a `getdata()` function to get its data from the user at the keyboard, and a `putdata()` function to display its data.

Write a `main()` program to test the `book` and `tape` classes by creating instances of them, asking the user to fill in data with `getdata()`, and then displaying the data with `putdata()`.

- *2. Recall the `STRCONV` example from Chapter 8. The `String` class in this example has a flaw: It does not protect itself if its objects are initialized to have too many characters. (The `SZ` constant has the value 80.) For example, the definition

```
String s = "This string will surely exceed the width of the "
         "screen, which is what the SZ constant represents.";
```

will cause the `str` array in `s` to overflow, with unpredictable consequences, such as crashing the system.

With `String` as a base class, derive a class `Pstring` (for “protected string”) that prevents buffer overflow when too long a string constant is used in a definition. A new constructor in the derived class should copy only `SZ-1` characters into `str` if the string constant is longer, but copy the entire constant if it’s shorter. Write a `main()` program to test different lengths of strings.